

Maximum Subarray Sum Project

Complete Presentation Speech

CSE 317: Algorithms – Design & Analysis

Spring 2026

M. Faraz Ansari (29201)

Areeshah Faseeh (29007)

Fatima Faisal (28989)

Marium Ali Lakhani (29183)

Khadeja Qureshi (27200)

Contents

1	Slide 1: Title Slide	3
2	Slide 2: The Problem	4
3	Slide 3: Algorithms at a Glance	5
4	Slide 4: Pseudocode – Part I	6
5	Slide 5: Pseudocode – Divide & Conquer	7
6	Slide 6: Theoretical Analysis	8
7	Slide 7: Empirical Results – Log-Log Plot	9
8	Slide 8: Speedup Chart	10
9	Slide 9: Runtime at $n = 3000$	11
10	Slide 10: Heatmap	12
11	Slide 11: Dataset Sensitivity – Kadane Anomaly	13
12	Slide 12: Theory vs. Empirical	14
13	Slide 13: Strengths & Weaknesses	15
14	Slide 14: Conclusion	16

Slide 1: Title Slide

Time: 30 seconds

Good morning everyone. Before we begin, let me quickly introduce our team – we have Areesha, Fatima, Marium, Khadeja, and Faraz here with me today.

Today we are going to walk you through our project on the Maximum Subarray Sum problem. The reason we chose this problem is because it is one of those rare problems that can be solved in many different ways. You can solve it using a simple but slow approach, or you can solve it using a clever and very fast approach. And the difference between these approaches is huge – we are talking about a difference of thousands or even hundreds of thousands of times in terms of speed.

So what did we actually do in this project? We implemented five different algorithms that all solve the exact same problem. And then we ran them on the same datasets, on the same machine, and we measured how long each one takes. Our goal was to see how algorithm design choices affect real-world performance.

The five algorithms we are comparing are: the Brute Force algorithm which is the simplest but slowest, the Better Brute Force which improves on it a little bit, the Prefix Sum method which uses a different technique, the Divide and Conquer approach which is more clever, and finally Kadane's Algorithm which is considered the optimal solution for this problem.

Each of these algorithms represents a different way of thinking about the same problem – from just checking everything possible, to splitting the problem into smaller pieces, to using dynamic programming. And by the end of this presentation, you will see just how much these different thinking styles matter when you actually run the code.

Transition: Let me now hand over to my teammate who will explain what the problem actually is.

Slide 2: The Problem

Time: 45 seconds

Thank you. So let me explain what the Maximum Subarray Sum problem actually asks us to do.

Imagine you are given an array of numbers. Some of these numbers are positive, some are negative, and some might be zero. Your job is to find a contiguous block of elements – meaning elements that are next to each other in the array – that gives you the largest possible sum. That is it. That is the whole problem.

Now let me show you an example to make this crystal clear. Look at this array: negative 2, 1, negative 3, 4, negative 1, 2, 1, negative 5, 4. Now you might think, why not just take all the numbers? If we add everything up, we get 1. But that is not the largest sum.

If we look carefully, the subarray starting at the number 4, then negative 1, then 2, then 1 – that block gives us a sum of 6. And if you try any other block, you will not find a sum larger than 6. So the answer for this array is 6.

Now you might be wondering – why does anyone care about this problem? It sounds like a puzzle, not something real. But actually, this problem appears in many real-world situations.

First, in finance. Imagine you have stock prices for each day, and you want to find the best time period to buy and sell. If you think of daily price changes as positive or negative numbers, then finding the maximum subarray sum tells you exactly the most profitable window.

Second, in signal processing. When you receive a signal with noise, you want to find the strongest part of the signal. That is basically finding the maximum subarray sum.

Third, in bioinformatics. When scientists analyze DNA sequences, they look for regions that have high scores for certain properties. Again, this is the maximum subarray problem.

And fourth, in image processing. If you want to find the brightest region in an image, you can extend this problem to two dimensions.

So this is not just a classroom exercise – it is a problem that people actually need to solve in the real world. And that is why understanding how to solve it efficiently matters.

Transition: Now let me pass it to my teammate who will give you a quick overview of all five algorithms we compared.

Slide 3: Algorithms at a Glance

Time: 30 seconds

Thank you. So before we dive into the details of each algorithm, let me give you a bird's eye view of what we are dealing with.

Looking at this table, you can see all five algorithms lined up. And the first thing you will notice is the complexity column. Look at how different they are.

The Brute Force algorithm at the top has complexity O of n cubed. That means if you double your input size, the running time increases by roughly eight times. That is very slow growth.

The Better Brute Force and the Prefix Sum method both have O of n squared. That is better than cubic, but still slow. If you double the input size, the time increases by about four times.

Then we have Divide and Conquer with O of $n \log n$. This is much better. If you double the input size, the time increases by a little more than double.

And finally, we have Kadane's Algorithm with O of n . This is the best possible for this problem because you have to look at every element at least once. So O of n is optimal – you cannot do better than that.

Now notice that each algorithm uses a different design philosophy. Brute Force uses exhaustive search. Better Brute Force is still exhaustive but with a small improvement. Prefix Sum uses precomputation. Divide and Conquer uses the split-and-combine approach. And Kadane uses dynamic programming.

Kadane's is the one I want you to pay special attention to. It is the theoretical champion – O of n time and O of 1 space. That means it is both fast and memory efficient. As we go through the rest of the presentation, you will see just how much faster it really is compared to the others.

Transition: Now let me hand over to my teammate who will walk you through the actual code for the first four algorithms.

Slide 4: Pseudocode – Part I

Time: 60 seconds

Thank you. Now let me explain how the first four algorithms actually work. I will go through them one by one.

Starting with the Brute Force algorithm – this is the simplest but also the slowest. How does it work? It uses three nested loops. The first loop picks a starting index. The second loop picks an ending index. And then the third loop adds up all the numbers from the start to the end. After calculating the sum, it checks if this is the largest sum it has seen so far, and if yes, it saves it. Because of these three loops, the time complexity is O of n cubed. The code is very straightforward, but the problem is that it does a lot of repeated work.

Now look at the Better Brute Force algorithm. This one removes that innermost loop. How? Instead of recalculating the sum each time, it keeps a running sum. Start from the starting index, add the next element, and keep adding as you go. This way, you only do two loops instead of three. The complexity becomes O of n squared. This is much better than O of n cubed, but still not great for large inputs.

Next is the Prefix Sum algorithm. This one takes a different approach. First, it creates a new array called prefix. At each position, prefix stores the sum of all elements from the beginning up to that point. Once you have this prefix array, any subarray sum becomes a simple subtraction. This makes the innermost calculation very fast – just one subtraction. But here is the catch – you still need two loops to check all possible pairs. So the overall complexity is still O of n squared. The advantage is that if you need to calculate many different subarray sums, the prefix array is reusable.

Now finally, let me explain Kadane's Algorithm. This one is beautiful in its simplicity. You start with the first element. Then you loop through the array once. At each step, you ask one simple question: is it better to continue the subarray we already have, or start a new subarray from this element? You answer this by taking the maximum between the current element alone and the current element plus the previous sum. Then you update your best answer if needed. That is it. One pass, one decision per element. That is why Kadane achieves O of n time and O of 1 space.

Transition: Now let me pass it to my teammate who will explain the Divide and Conquer algorithm in more detail.

Slide 5: Pseudocode – Divide & Conquer

Time: 45 seconds

Thank you. Now let me explain the Divide and Conquer algorithm. This one is more complex but also very elegant.

The idea behind Divide and Conquer is simple: break the big problem into smaller problems, solve those, and then combine the results. For maximum subarray, it works like this.

First, you find the middle of the array. Then you split the array into a left half and a right half. Now here is the key insight – the maximum subarray of the whole array must be one of three things. It might be completely inside the left half. It might be completely inside the right half. Or it might cross the middle, meaning it includes some elements from the left and some from the right.

So you recursively find the maximum subarray in the left half. Then you recursively find the maximum subarray in the right half. Then you separately find the maximum subarray that crosses the middle.

How do you find the crossing subarray? You start from the middle and move leftwards, adding elements as you go, and you keep track of the largest sum you see on the left side. Then you start from the middle plus one and move rightwards, doing the same thing. Then you add the best left sum and the best right sum together – that gives you the best crossing subarray.

Finally, you take the maximum of these three: left best, right best, and crossing best. That becomes your answer for that part of the array.

Because you split the array in half at each level, and you do linear work at each level to find the crossing sum, the recurrence is T of n equals 2 times T of n over 2 plus O of n . Using the Master Theorem, this solves to O of $n \log n$.

So to summarize: Divide means split the array in half. Conquer means recursively solve each half. Combine means find the crossing subarray and pick the best of all three.

Transition: Now let me pass it to my teammate who will show you the theoretical analysis of all these algorithms.

Slide 6: Theoretical Analysis

Time: 30 seconds

Thank you. Now let me show you what the theory predicts before we look at our actual measurements.

This chart shows the theoretical operation counts for each algorithm at different input sizes. What I want you to notice is how quickly the lines separate from each other.

Take n equals 1000 as an example. The Brute Force algorithm needs about 1 billion operations. Kadane's Algorithm needs only 1000 operations. That is a difference of one million times. And look at what happens as n gets larger. At n equals 3000, Brute Force needs 27 billion operations, while Kadane still only needs 3000 operations. The gap just keeps growing.

This is why we say that complexity matters. It is not about writing code that works – any of these algorithms will give you the correct answer. It is about writing code that finishes. If you are processing a large dataset, the difference between O of n cubed and O of n could be the difference between waiting a few milliseconds and waiting for days.

The theoretical analysis gives us a baseline. It tells us what we should expect to see when we run our actual code. If our measurements match these predictions, then we know our implementations are correct and our benchmarking is reliable.

Transition: Now let me pass it to my teammate who will show you what we actually measured when we ran these algorithms.

Slide 7: Empirical Results – Log-Log Plot

Time: 45 seconds

Thank you. This slide is one of the most important ones because it shows what we actually measured when we ran the code.

First, let me explain what a log-log plot is, because it might not be familiar to everyone. On a normal graph, if you plot O of n squared, it curves upward very steeply. But if you take the logarithm of both the x-axis and the y-axis, that curve becomes a straight line. And the slope of that straight line tells you the exponent in the complexity.

So on this graph, the x-axis is the input size on a log scale. The y-axis is the execution time in milliseconds on a log scale. Each line represents one algorithm.

Now look at the Brute Force line – it has a slope of about 3. That matches O of n cubed. The Better Brute Force and Prefix Sum lines both have slopes of about 2. That matches O of n squared. The Divide and Conquer line has a slope a little above 1 – about 1.1 – which matches O of $n \log n$. And Kadane's line has a slope of almost exactly 1, which matches O of n .

Why is this important? Because the fact that our measured slopes match the theoretical predictions tells us two things. First, our implementations are correct – we did not accidentally make any algorithm faster or slower than it should be. Second, our benchmarking methodology is sound – we measured things accurately and fairly.

This alignment between theory and practice is really the heart of algorithm analysis. It shows that the mathematical models we learn in class actually predict real-world behavior. And that is very satisfying to see.

Transition: Now let me pass it to my teammate who will show you just how big the speed differences actually are.

Slide 8: Speedup Chart

Time: 30 seconds

Thank you. Now let me put these differences in concrete terms that are easy to understand.

What is speedup? It simply means how many times faster one algorithm is compared to another. Here we are comparing every algorithm to the Brute Force baseline.

At n equals 3000, which is the largest size we tested, look at what we found. The Better Brute Force algorithm is about 450 times faster than Brute Force. The Prefix Sum algorithm is also about 450 times faster. The Divide and Conquer algorithm is about 27 thousand times faster. And Kadane's Algorithm is about half a million times faster.

Let me repeat that because it sounds unbelievable – Kadane's Algorithm is approximately 500,000 times faster than Brute Force at n equals 3000.

To give you a mental picture: if Brute Force takes 14 seconds, Kadane takes 0.025 milliseconds. That is like comparing a car traveling across the country to a car moving one inch. Or like comparing reading an entire library to reading a single sentence.

And here is the key point – this speedup only gets larger as n increases. The gap between O of n cubed and O of n grows without bound. So for very large datasets, the difference becomes even more dramatic.

This slide really drives home why algorithm choice matters. It is not about saving a few seconds here and there. It is about whether your program finishes at all for large inputs.

Transition: Now let me pass it to my teammate who will show you the actual numbers we measured.

Slide 9: Runtime at $n = 3000$

Time: 30 seconds

Thank you. Now let me give you the actual numbers from our measurements on random data.

At n equals 3000, the Brute Force algorithm took about 13,766 milliseconds. That is almost 14 seconds. If you were waiting for an answer, you would definitely notice that delay.

The Better Brute Force took about 31.7 milliseconds. That is about one thirtieth of a second – you would barely notice it.

The Prefix Sum took about 30.7 milliseconds – almost identical to Better Brute Force, as we expected.

The Divide and Conquer algorithm took about 0.47 milliseconds. That is less than half of one millisecond – too fast for a human to perceive.

And Kadane's Algorithm took about 0.025 milliseconds. That is 25 microseconds – essentially instant.

Now look at the bar chart on the screen. The Brute Force bar is so tall that all the other bars look like they are at zero. That is why we use a log scale for most of our graphs – so you can actually see all the algorithms on the same chart.

One more important thing to mention: before we ran any of these timing tests, we verified that all five algorithms produce the exact same answers. We used a small test case that we all know the answer to – the example array I showed you earlier – and every algorithm gave the correct sum of 6. We also tested on all-negative arrays, all-positive arrays, and random arrays. In every case, all five algorithms agreed. So the comparison is fair – they all solve the problem correctly, just at very different speeds.

Transition: Now let me pass it to my teammate who will show you a different view of the same data.

Slide 10: Heatmap

Time: 30 seconds

Thank you. This slide shows the same data we just discussed, but in a different visual format called a heatmap.

In a heatmap, darker colors mean slower execution times, and lighter colors mean faster times. This makes it very easy to see patterns at a glance.

Look at the Brute Force column – it is completely dark for every input size. From n equals 100 all the way to n equals 3000, it is the darkest color on the scale. That tells you that Brute Force is always the slowest by a huge margin.

Now look at the Divide and Conquer and Kadane columns – they are very light, almost white. That tells you they are very fast across all input sizes.

The Better Brute Force and Prefix Sum columns are somewhere in the middle – not as dark as Brute Force, but not as light as Kadane.

Also notice how the color changes as you go down the rows. For Brute Force, it gets darker very quickly as n increases. For Kadane, it barely changes at all. This visual difference captures the complexity difference perfectly.

The heatmap is useful because it summarizes a lot of data in a single image. You can see all five algorithms, all five input sizes, and all three dataset types in one place. It really drives home the point that algorithm choice is the single biggest factor in performance.

Transition: Now let me pass it to my teammate who will talk about an interesting anomaly we found in our data.

Slide 11: Dataset Sensitivity – Kadane Anomaly

Time: 45 seconds

Thank you. Now I want to point out something interesting we found in our data – a small anomaly that actually teaches us something about how computers work.

Look at this graph. It shows Kadane’s Algorithm performance across different dataset types. For random data and positive data, the line is smooth – time increases slowly and steadily as n grows. That is what we expect from an O of n algorithm.

But now look at the negative-only dataset. At n equals 500, there is a small spike. The time jumps from about 0.01 milliseconds to about 0.2 milliseconds. That is a 20 times increase for just that one data point. Then at n equals 1000, it drops back down.

So what happened? Why would an O of n algorithm suddenly get slower for a specific input size on a specific dataset type?

The answer has to do with something called branch prediction. Modern processors try to guess what your code will do next so they can prepare in advance. When the guess is correct, everything runs fast. When the guess is wrong, the processor has to pause and correct itself, which takes extra time.

In Kadane’s algorithm, there is an implicit branch – the decision to either continue the current subarray or start a new one. On random data, this decision pattern is random, so the branch predictor does not have a strong expectation. On positive data, you almost always continue, so the branch predictor learns and becomes accurate.

But on negative data, something strange happens. When all numbers are negative, the algorithm constantly chooses to start a new subarray because adding a negative number to an already negative sum makes it even worse. This pattern might cause a specific type of misprediction at certain array sizes.

The exact reason for the spike at exactly n equals 500 could be cache alignment, or branch predictor history length, or some other hardware detail. But the key takeaway is this: even an optimal algorithm can show small variations depending on the data. However, notice that the spike is still small – 0.2 milliseconds is still extremely fast. And most importantly, the algorithm remains O of n . The complexity class did not change – only the constant factor increased slightly for that specific case.

Transition: Now let me pass it to my teammate who will summarize how our empirical results compare to the theoretical predictions.

Slide 12: Theory vs. Empirical

Time: 30 seconds

Thank you. This slide is our validation slide. It shows side by side what we expected from theory and what we actually measured.

For each algorithm, we have what we expected and what we measured. Now look at the results.

Brute Force – theoretical O of n cubed, empirical slope about 2.99 – that is essentially 3. They match.

Better Brute Force – theoretical O of n squared, empirical slope about 2.0. They match.

Prefix Sum – theoretical O of n squared, empirical slope about 2.0. They match.

Divide and Conquer – theoretical O of $n \log n$, empirical slope about 1.1. That matches because $\log n$ grows very slowly, so $n \log n$ looks almost linear on a log-log plot.

Kadane – theoretical O of n , empirical slope about 1.0. They match.

So all five algorithms show the expected complexity. This is a very clean result. It tells us that our implementations are faithful to the algorithms – we did not accidentally make any of them faster or slower than they should be. And it tells us that our benchmarking methodology is reliable – we measured what we intended to measure.

This alignment between theory and practice is what algorithm analysis is all about. The mathematical models we learn actually predict real behavior, and that is very satisfying to see in actual data.

Transition: Now let me pass it to my teammate who will talk about the strengths and weaknesses of each algorithm.

Slide 13: Strengths & Weaknesses

Time: 45 seconds

Thank you. Before we conclude, I want to be fair to all the algorithms. Kadane is the fastest, but that does not mean the other algorithms are useless. Each one has situations where it might be the right choice.

Let me go through them one by one.

First, Brute Force. Its strength is that it is extremely simple to understand and implement. If you are just learning programming, you can write this algorithm in a few minutes. Also, because it is so simple, it is very easy to verify that it is correct. The weakness is that it is O of n cubed, which makes it completely unusable for any input larger than a few thousand elements.

Second, Better Brute Force. Its strength is that it is still very simple, but it improves the complexity to O of n squared. For moderate input sizes, this might be acceptable. The weakness is that O of n squared still becomes slow when n gets large.

Third, Prefix Sum. Its strength is that the prefix array can be reused. If you need to calculate many different subarray sums for the same array, this method is efficient. Also, this approach extends naturally to two dimensions. The weakness is that it still takes O of n squared to find the maximum subarray if you do not know where to look, and it uses extra memory.

Fourth, Divide and Conquer. Its strength is that it achieves O of $n \log n$ time, which is very good. Also, because the left and right halves are independent, this algorithm can be parallelized easily. The weakness is that it is more complex to implement correctly, and the recursion uses stack space.

Fifth, Kadane's Algorithm. Its strengths are clear – it is O of n time and O of 1 space, making it both fast and memory efficient. The code is simple – less than 10 lines. And it is cache-friendly because it makes one sequential pass through memory. The weaknesses are minor. It only works for the one-dimensional version of the problem. Also, as we saw, it can have small performance variations depending on the data due to branch prediction.

So the takeaway here is that there is no single best algorithm for every situation. If you need a quick solution for a one-dimensional array and performance matters, use Kadane. If you are learning or need to verify correctness, start with Brute Force. If you need to parallelize, consider Divide and Conquer.

Transition: Now let me pass it to my teammate for our conclusion.

Slide 14: Conclusion

Time: 45 seconds

Thank you everyone for listening. Let me now summarize what we have learned from this project with four main takeaways.

First, theory matches practice. We derived the theoretical complexities of all five algorithms, we implemented them correctly, and when we measured their performance, the empirical slopes on our log-log plots matched the theoretical predictions perfectly. This alignment confirms that the mathematical models we learn in algorithm analysis are not just abstract ideas – they predict real-world behavior.

Second, Kadane's Algorithm dominates. At n equals 3000, Kadane was about 500,000 times faster than Brute Force. That is not a small improvement – that is a difference between waiting almost 14 seconds versus waiting 25 microseconds. This is why we say that choosing the right algorithm matters more than buying faster hardware or micro-optimizing your code.

Third, algorithmic paradigm matters. Each algorithm we studied represents a different way of thinking about the same problem. Brute Force is exhaustive search. Prefix Sum is precomputation. Divide and Conquer is recursive splitting. Kadane is dynamic programming. Each paradigm produced measurably different performance. The design choices you make as a programmer directly affect how your program runs.

Fourth, even optimal algorithms can show small dataset sensitivities. We saw a small spike on Kadane's algorithm with all-negative data at n equals 500. This was caused by branch misprediction – a hardware-level effect. But notice that even with this spike, Kadane remained O of n – the complexity class did not change.

So what is the final takeaway from this project? Understanding algorithm design is not just academic. It directly determines whether your program finishes in milliseconds or hours. The difference between a cubic algorithm and a linear algorithm on a large dataset is the difference between a solution that works and a solution that is practically useless.

We hope this presentation has given you a clear picture of how different algorithms for the same problem can lead to wildly different performance, and why it is worth taking the time to choose the right approach.

Thank you for your attention. We are now happy to take any questions you might have.

Appendix: Possible Viva Questions and Answers

Q1: Why did you choose the Maximum Subarray Sum problem for this project?

Answer: We chose this problem because it can be solved using multiple different algorithm design paradigms – brute force, divide and conquer, and dynamic programming. Each approach gives a very different time complexity, from $O(n^3)$ down to $O(n)$. This makes it perfect for comparing how algorithm design choices affect real performance. Also, the problem has real-world applications in finance, signal processing, and bioinformatics.

Q2: What is the main difference between Kadane's Algorithm and the Divide and Conquer approach?

Answer: The main difference is in how they solve the problem. Kadane's Algorithm uses dynamic programming – it makes one pass through the array, at each step deciding whether to continue the current subarray or start a new one. This gives $O(n)$ time. Divide and Conquer splits the array in half, recursively solves each half, and then finds the crossing subarray. This gives $O(n \log n)$ time. Kadane is faster and uses less memory, but Divide and Conquer has the advantage that it can be parallelized more easily.

Q3: How did you ensure fair comparison between all algorithms?

Answer: We used three main methods to ensure fairness. First, same datasets – all algorithms ran on identical datasets with the same random seeds. Second, same hardware – all tests were run on the same machine with the same compiler flags. Third, verification step – before benchmarking, we ran a correctness check to confirm all algorithms produced identical results on small test cases.

Q4: Explain the anomaly you saw with Kadane's algorithm on negative-only data.

Answer: On the all-negative dataset at $n=500$, Kadane's execution time spiked from about 0.01ms to 0.2ms. This happens because when all numbers are negative, Kadane constantly resets its current sum. This pattern of repeated resets confuses the CPU's branch predictor, causing mispredictions which slow down the code slightly. However, even with this spike, the algorithm is still $O(n)$ – the complexity class did not change.

Q5: Why is Brute Force $O(n^3)$ and not $O(n^2)$?

Answer: Brute Force uses three nested loops. The first loop sets the starting index, the second loop sets the ending index, and the third loop calculates the sum from start to end. The number of start-end pairs is $O(n^2)$, and for each pair, the sum calculation takes $O(n)$ in the worst case. Multiplying gives $O(n^3)$. Better Brute Force removes that innermost loop by keeping a running sum, so it becomes $O(n^2)$.

Q6: Can Kadane's algorithm handle an array with all negative numbers?

Answer: Yes, it handles it correctly. When all numbers are negative, Kadane's algorithm at each step compares the current element with current sum plus the element. Since current sum is negative, adding it makes the sum even more negative, so the algorithm chooses just the current element. This way, it effectively picks the largest (least negative) number in the array. The algorithm still runs in $O(n)$ time.

Q7: What would happen if you ran these tests on much larger input sizes?

Answer: For $n = 1$ million, Brute Force would be impossible (about 10^{18} operations), Better Brute Force would take days or weeks (about 10^{12} operations), Divide and Conquer would take seconds (about 20 million operations), and Kadane would take milliseconds (about 1 million operations). The gap would become even larger.

Q8: How did you generate your test datasets?

Answer: We generated three types of datasets for each input size. Random datasets with values between -1000 and 1000 using a fixed random seed of 42. All-negative datasets with values between -1000 and -1 using seed 99. All-positive datasets with values between 1 and 1000 using seed 7. We used fixed seeds so that anyone can reproduce our exact results.

Q9: What is branch misprediction and why did it affect your results?

Answer: Modern CPUs try to guess which way a branch will go to keep the pipeline full. When the guess is wrong, there is a small penalty. In Kadane's algorithm, the decision is whether to continue the subarray or start over. On all-negative data, the answer is always "start over" – but the constant resets cause overhead. The spike we saw at $n=500$ is a combination of these factors.

Q10: Which algorithm would you recommend for a real-world application and why?

Answer: For the 1D Maximum Subarray problem, we recommend Kadane's Algorithm because it is optimal with $O(n)$ time and $O(1)$ space, it is simple to implement in less than 10 lines of code, it works for all input types, and it is cache-friendly. The only reasons to consider another algorithm would be if you needed to extend to 2D problems, or if you needed a parallelizable solution.

Q11: How did you measure execution time precisely?

Answer: In C++, we used `std::chrono::high_resolution_clock` which provides microsecond or even nanosecond precision. We took measurements right before and right after each algorithm call. To reduce noise, we ran each test multiple times. We also compiled with `-O0` to disable compiler optimizations that might unfairly speed up one algorithm over another.

Q12: What was the most surprising finding in your project?

Answer: The most surprising finding was how big the difference actually is between algorithms. Seeing a 500,000x speedup at $n=3000$ was still shocking even though we knew Kadane would be faster. Also, the branch misprediction spike on Kadane with negative data was unexpected – it shows that even optimal algorithms can have interesting hardware-level interactions.

Q13: If you had more time, what would you do next?

Answer: We would extend the analysis to the 2D Maximum Subarray problem, test on much larger input sizes up to 10 million, add Python implementations to compare language speeds, analyze cache behavior and memory access patterns, and implement parallel versions of Divide and Conquer.

Q14: How do you know your implementations are correct?

Answer: We ran a verification step before any benchmarking. All five algorithms processed the same known test cases, including the example array that should return 6. All algorithms gave the same answer. We also tested on edge cases like single-element arrays and all-negative arrays. Only after confirming correctness did we run the benchmarks.

Q15: What is the practical takeaway from your project?

Answer: The practical takeaway is simple – choose the right algorithm for your problem. A naive solution might work for small inputs, but as data grows, algorithm choice becomes critical. Kadane’s Algorithm is only 10 lines of code but runs 500,000 times faster than brute force. Understanding algorithm design directly determines whether your application runs in milliseconds or hours.